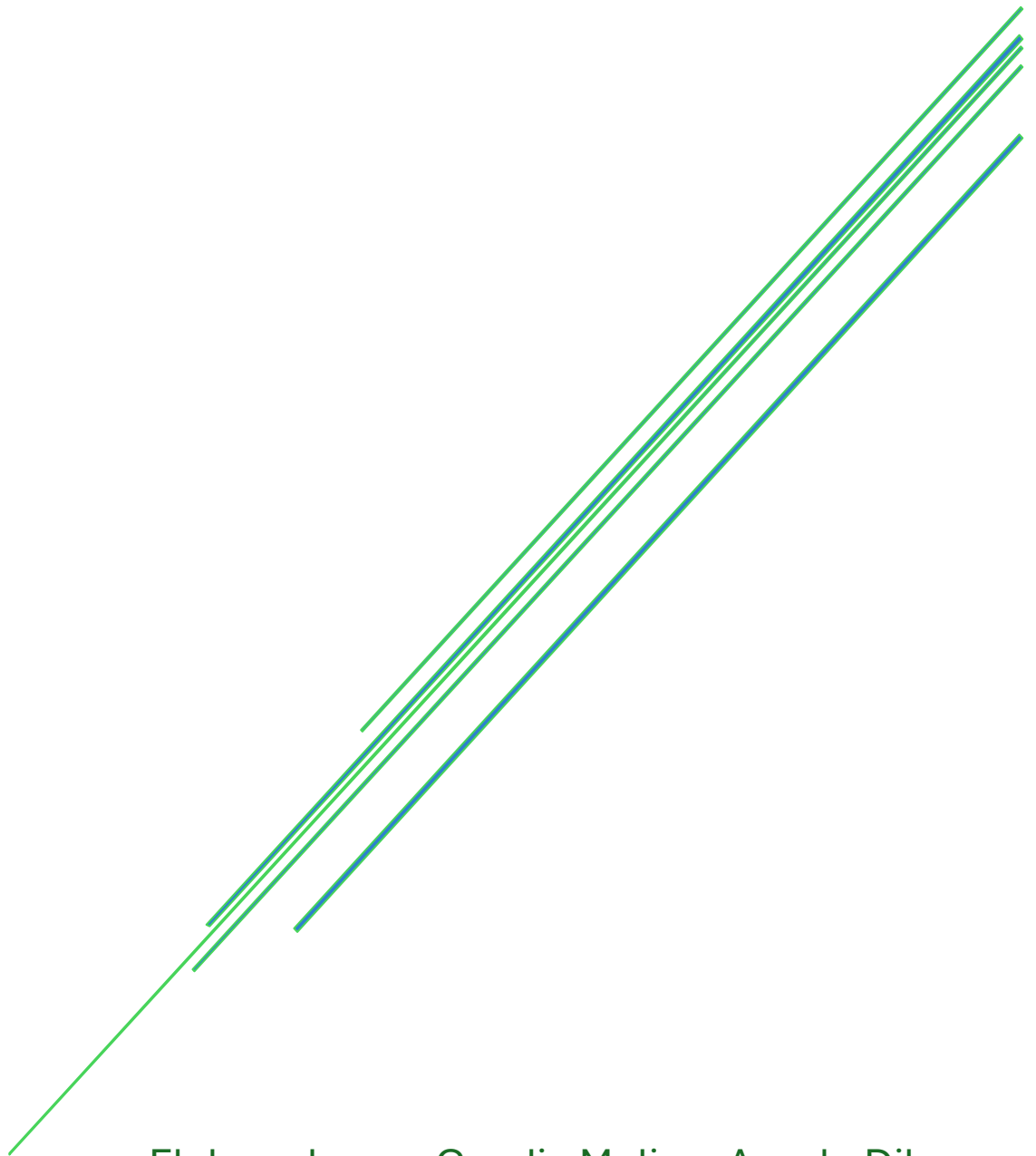


PROCESAMIENTO DE IMÁGENES

Expansión de un histograma



Elaborado por: Candia Molina, Anyelo Dilan
Matemática computacional

Contenido

1. Objetivos.....	4
2. Fundamentos Teóricos.....	4
2.1. Histograma	4
2.2. Histograma de una imagen	4
2.3. Expansión de un Histograma	5
3. Aplicaciones.....	6
4. Programa	7
4.1. Expansión de un histograma en Python	7
Conclusiones	16
Referencias Bibliográficas	17
ANEXOS.....	17

Ecualización de imágenes

La ecualización de imágenes es una técnica fundamental del procesamiento digital de imágenes que se emplea para mejorar el contraste, permitiendo una mejor visualización de los detalles presentes en una imagen. Esta técnica resulta especialmente útil cuando la imagen es muy oscura, muy clara o presenta una distribución limitada de niveles de gris, ya que redistribuye las intensidades para aprovechar todo el rango dinámico disponible.

Soy estudiante de la carrera de Ingeniería de Sistemas de la Información en la Universidad Peruana de Ciencias Aplicadas y me he propuesto desarrollar códigos con base matemática que puedan servir como apoyo y guía para otros estudiantes interesados en el aprendizaje del procesamiento digital de imágenes. Considero que poseo una buena lógica de programación; sin embargo, reconozco que esta puede ser mejorada, ya que en ocasiones realizo procesos innecesarios que podrían optimizarse mediante una mejor estructuración del código.

El objetivo principal de este proyecto es comprender y aplicar los conceptos matemáticos involucrados en la ecualización y expansión del histograma, utilizando programación básica para implementar dichas técnicas. De esta manera, se busca fortalecer la relación entre la teoría matemática y su aplicación práctica, facilitando el aprendizaje progresivo de los fundamentos del procesamiento de imágenes digitales.

1. Objetivos

Desarrollar un programa en Python que pueda mejorar la calidad de imágenes mediante la ecualización de histogramas.

Para desarrollar el programa en un 100% debemos cumplir los siguientes puntos.

- Demostrar la comparación de histogramas e imágenes.
- Subida de imágenes mediante botones.

2. Fundamentos Teóricos

2.1. Histograma

El histograma es un diagrama de barras cuyas abscisas representan los niveles de gris de una imagen, y las ordenas, las frecuencias relativas de los distintos niveles de gris (Fernando et al, 2005).

$$P_K = \frac{n_K}{n}$$

Donde:

n_K = cantidad de pixeles de nivel de gris k

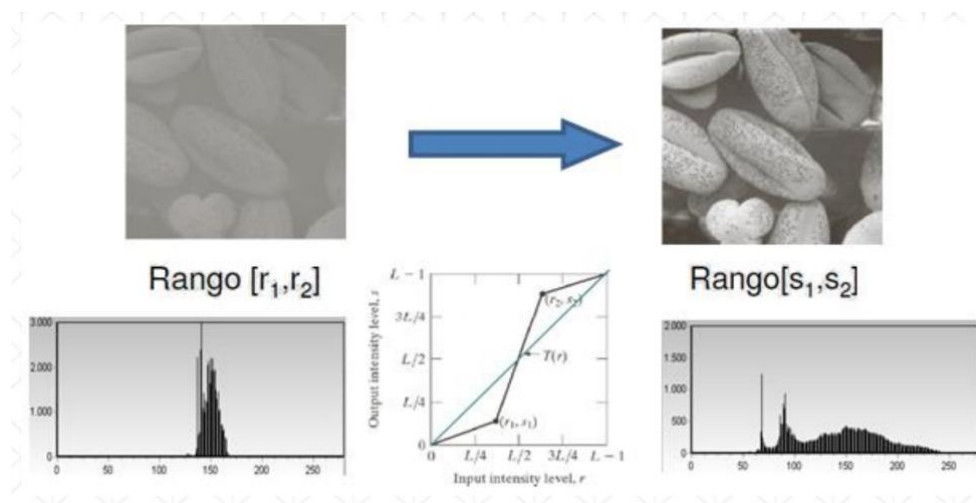
n = cantidad total de pixeles

2.2. Histograma de una imagen

Este proceso consiste en aumentar el rango de niveles de gris de la imagen.

Figura 1

Ejemplos de expansión de imágenes



Nota: Sacado del material de la Universidad Peruana de Ciencias Aplicadas.

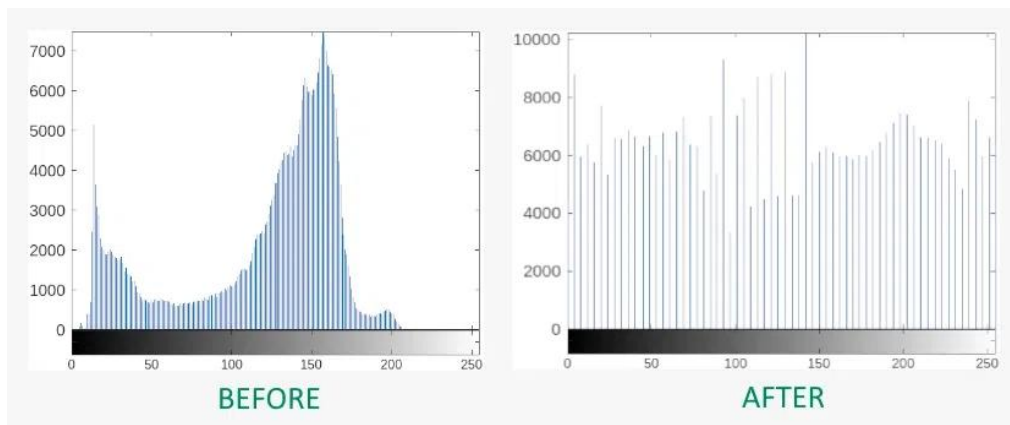
2.3. Expansión de un Histograma

La expansión de histograma (también llamada *estiramiento de contraste*) es una técnica de procesamiento de imágenes cuyo objetivo es aumentar el contraste de una imagen ampliando el rango de niveles de gris que utiliza.

La expansión del histograma consiste en reescalar esos valores para que el nivel mínimo se acerque al negro y el máximo al blanco, aprovechando todo el rango posible.

Figura 2

Expansión de un histograma

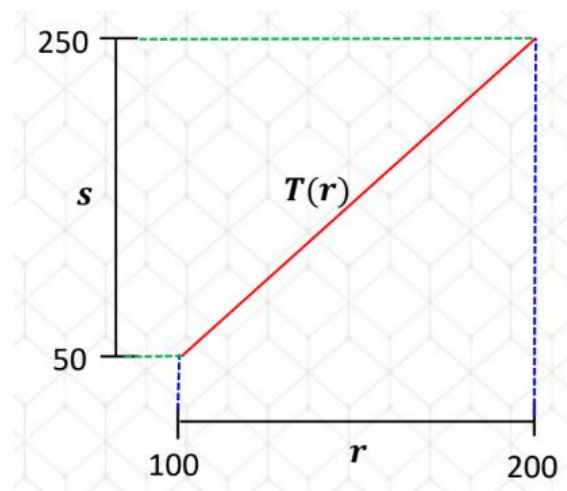


Nota: Para expandir un histograma a un intervalo mayor usaremos propiedades del Cálculo mediante una función lineal.

Transformación " $T(r)$ "

Figura 3

Grafica de la función de transformación



Nota: Sacado del material de la Universidad Peruana de Ciencias Aplicadas.

Función de transformación:

$$S = T(r) = mr + b$$

Donde:

$T(r)$ = función de transformación

r = Intervalo inicial

s = Intervalo final

m = Factor de escala

b = Desplazamiento vertical de $T(r)$.

Entonces podemos decir que el contraste de la imagen aumentado y disminuyendo si la pendiente es mayor o menor a 1.

Si $m > 1$, el contraste aumenta.

Si $m < 1$, el contraste disminuye.

En términos prácticos, la pendiente controla la intensidad del estiramiento del histograma.

3. Aplicaciones

Medicina: El procesamiento de imágenes se utiliza en el análisis de imágenes médicas como radiografías, tomografías y resonancias magnéticas para mejorar el contraste, detectar anomalías y apoyar el diagnóstico de enfermedades.

Seguridad y vigilancia: Se aplica en sistemas de videovigilancia para el reconocimiento facial, detección de objetos y seguimiento de personas o vehículos, ayudando a mejorar la seguridad en espacios públicos y privados.

Visión por computadora en la industria: En procesos industriales, el procesamiento de imágenes permite inspeccionar productos, detectar defectos y controlar la calidad de manera automática, reduciendo errores humanos y optimizando la producción.

Agricultura: El procesamiento de imágenes se emplea para analizar imágenes de cultivos, detectar plagas, evaluar el estado de las plantas y estimar la productividad agrícola, contribuyendo a una gestión más eficiente de los recursos y a la toma de decisiones oportunas.

4. Programa

Para el desarrollo del programa usaremos Python. Primero deberemos descargar la biblioteca “cv2” la cual nos permitirá leer las imágenes que tenemos.

4.1. Expansión de un histograma en Python

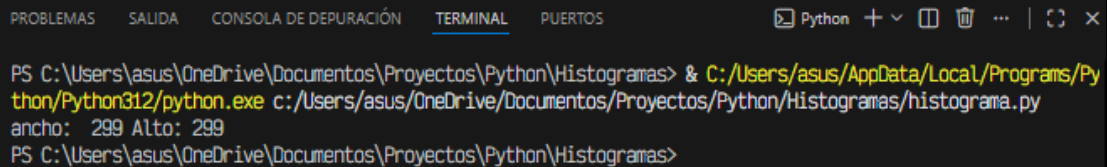
Paso 1: Deberemos obtener las dimensiones de la imagen.

```
import cv2

def Main():
    imagen = cv2.imread("pina.jpg")
    alto, ancho, canales = imagen.shape
    print(ancho, alto)

#principal
Main()
```

Resultado:

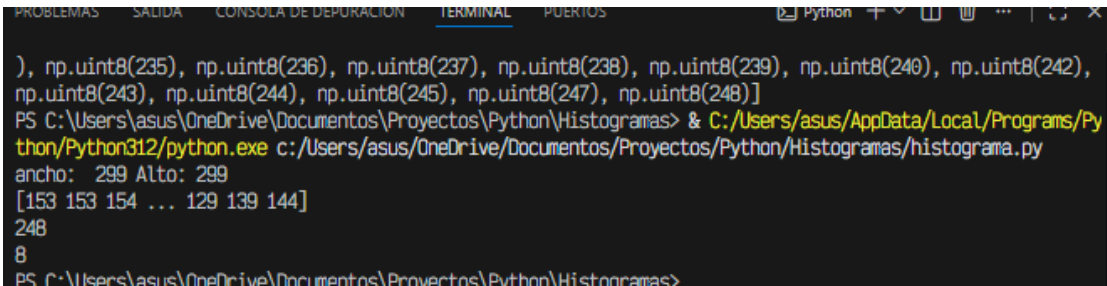


```
PS C:\Users\asus\OneDrive\Documentos\Proyectos\Python\Histogramas> & C:/Users/asus/AppData/Local/Programs/Python/Python312/python.exe c:/Users/asus/OneDrive/Documentos/Proyectos/Python/Histogramas/histograma.py
ancho: 299 Alto: 299
PS C:\Users\asus\OneDrive\Documentos\Proyectos\Python\Histogramas>
```

Paso 2: Deberemos obtener la cantidad de escala de grises de la imagen. En caso de tener una imagen de colores la convertiremos en gris y después tener una lista con las escalas de grises que tiene esta.

```
gris = cv2.cvtColor(imagen, cv2.COLOR_BGR2GRAY)
niveles_gris = gris.flatten()
print(niveles_gris)
print(max(niveles_gris))
print(min(niveles_gris))
print(list(set(niveles_gris)))
```

Resultado:



```
), np.uint8(235), np.uint8(236), np.uint8(237), np.uint8(238), np.uint8(239), np.uint8(240), np.uint8(242),
np.uint8(243), np.uint8(244), np.uint8(245), np.uint8(247), np.uint8(248)]
PS C:\Users\asus\OneDrive\Documentos\Proyectos\Python\Histogramas> & C:/Users/asus/AppData/Local/Programs/Python/Python312/python.exe c:/Users/asus/OneDrive/Documentos/Proyectos/Python/Histogramas/histograma.py
ancho: 299 Alto: 299
[153 153 154 ... 129 139 144]
248
8
PS C:\Users\asus\OneDrive\Documentos\Proyectos\Python\Histogramas>
```


Paso 3: Debemos obtener la cantidad de veces que se repite un píxel para ello, usaremos un diccionario.

Después mandar a imprimir el histograma con los valores obtenidos en el diccionario.

```
def cantidad_pixeles_color(lista):
    elementos = {}
    for i in lista:
        if i not in elementos:
            elementos[i] = 0

        if i in elementos.keys():
            elementos[i] += 1

    return elementos

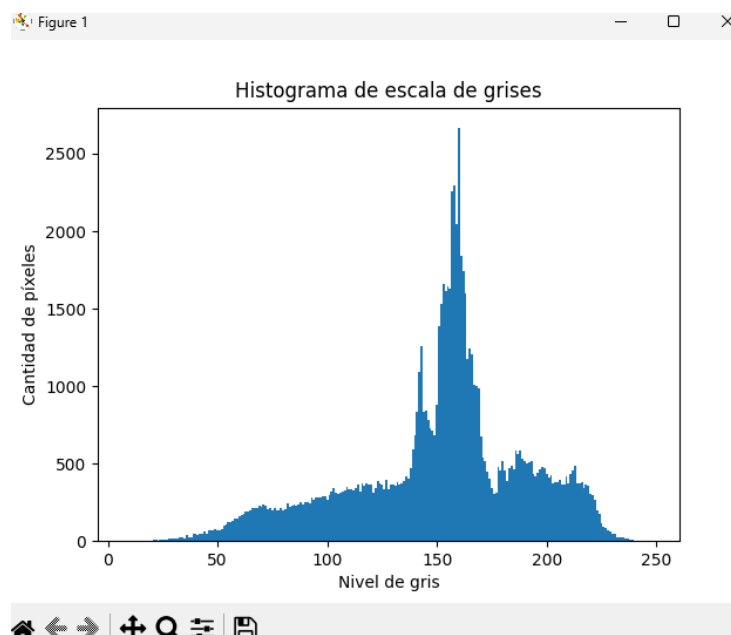
def dibujar_histograma(pixeles):
    niveles = list(pixeles.keys())
    cantidades = list(pixeles.values())

    plt.bar(niveles, cantidades, width=1)

    plt.title("Histograma de escala de grises")
    plt.xlabel("Nivel de gris")
    plt.ylabel("Cantidad de píxeles")
    plt.show()
```

Figura 4

Histograma de escala de grises de la imagen pina.jpg



Nota: Elaborado con Python.

Paso 4: Ordenaremos el histograma de escala mínima a máxima.

Antes deberemos tomar en cuentas los siguientes puntos

Puntos por considerar

- Debemos saber que blanco absoluto es 255
- Debemos saber que negro absoluto es 0
- Para expandir el histograma deberemos usar la función de expansión

Figura 5

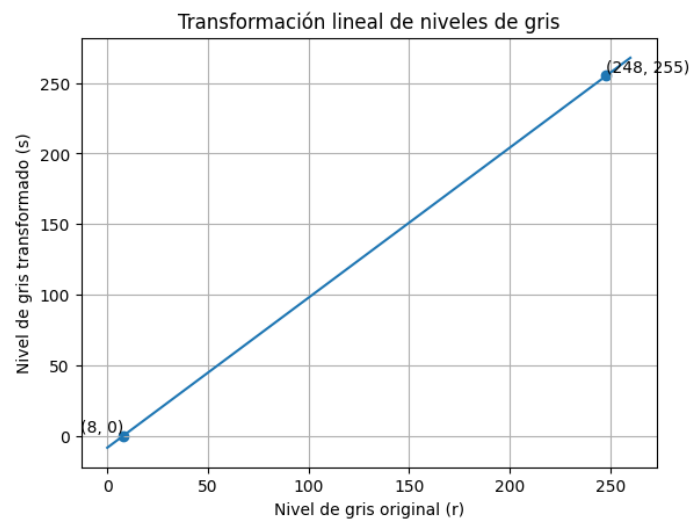
pina.jpg, 294x294 píxeles con escala de grises de [8; 248]



Nota: Elaborado con Gemini IA.

Figura 6

Gráfica de expansión de un histograma.



Nota: Elaboración propia.

Cálculos

Los puntos de la función son los límites de ambas escalas

Escala inicial: [8: 248]

Escala de expansión: [0: 255]

Sabemos:

$$S = T(r) = mr + b$$

Cálculo de la pendiente:

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

$$m = \frac{255 - 0}{248 - 8} = 1,0625$$

Cálculo de la constante:

$$y_1 = m \times x_1 + b$$

$$b = y_1 - (m \times x_1)$$

$$b = 0 - (1,0625 \times 8) = -8,5$$

Finalmente

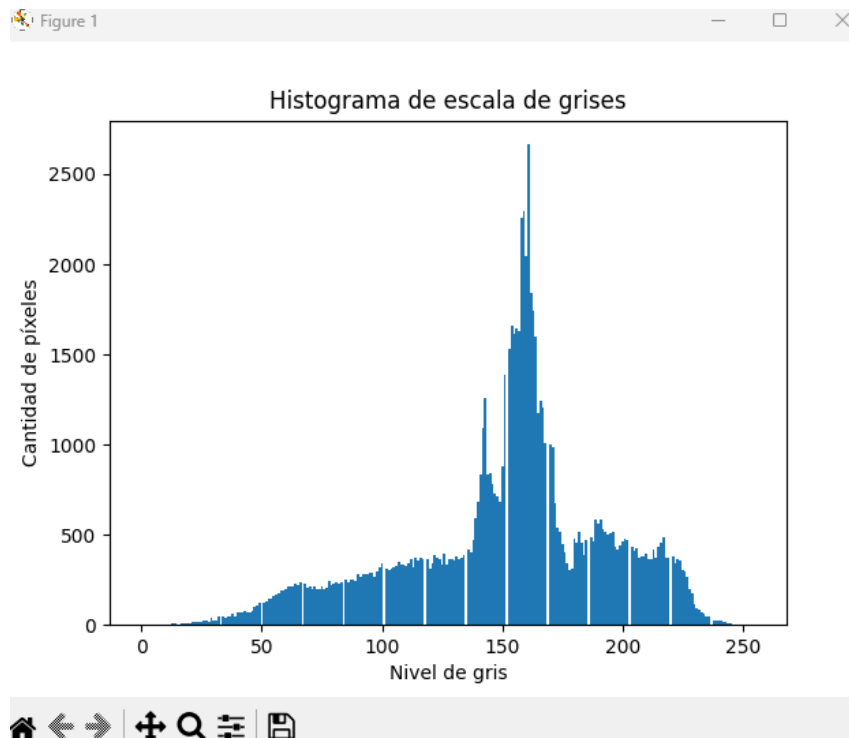
$$T(r) = 1,0625 \times r - 8,5$$

Tabulamos los valores de la escala anteriores para obtener las nuevas escalas

Transformación	Resultado
$T(8)$	0
$T(9)$	$1,0625 \cong 2$
$T(20)$	$12,750 \cong 13$
$T(100)$	$97,750 \cong 98$
$T(210)$	$214,625 \cong 215$
$T(248)$	255

Hay que recordar que cuando una escala de gris cambia entonces la cantidad de pixeles de esta deberán estar en la nueva escala de gris.

Figura 7

Histograma expandido*Nota:* Elaborado con Python.

```
def reordenamiento(inicial, final):
    punto1 = (inicial[0], final[0]) #interpretalo como (x, y)
    punto2 = (inicial[1], final[1])
    pendiente = (punto2[1] - punto1[1]) / (punto2[0] - punto1[0])
    constante = final[0] - (pendiente * inicial[0])
    return (pendiente, constante)

#----- transformación-----#
def funcion_transformacion(valores, gris):
    #De la forma T(r) = m * r + b
    new_valor = int((valores[0] * gris) + valores[1])
    return new_valor

def obtener_nuevos_valores(valores, pixeles):
    new_elementos = {}
    for i in pixeles:
        trans = funcion_transformacion(valores, i)
        new_elementos[trans] = pixeles[i]
    return new_elementos
```

Resultado:

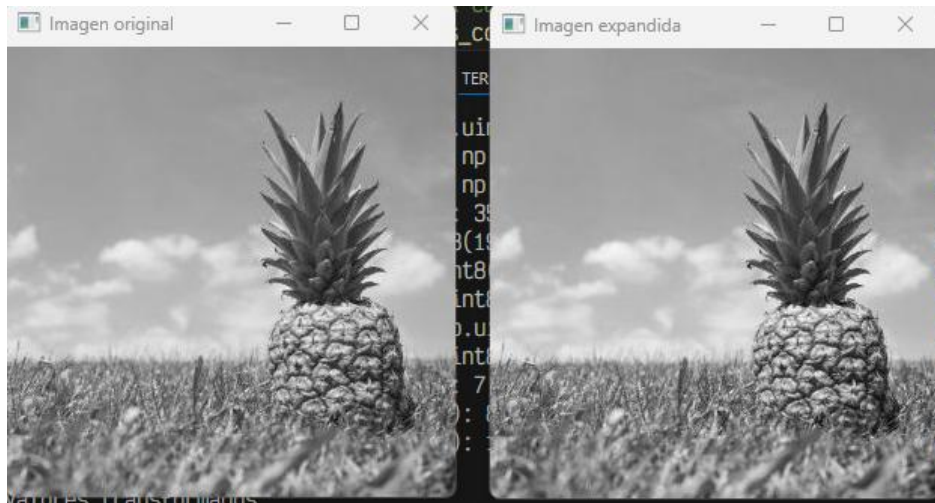
```
Valores transformados
-----
(np.float64(1.0625), np.float64(-8.5))
{154: 1663, 155: 1617, 153: 1530, 156: 1645, 157: 1628, 158: 2261, 151: 1385, 149: 681, 150: 880, 148: 716, 147: 732, 146: 784, 145: 841, 144: 836, 143: 1262, 142: 1
81, 139: 582, 138: 469, 137: 402, 136: 420, 134: 389, 133: 371, 131: 382, 132: 363, 159: 2298, 160: 2049, 161: 2665, 162: 1845, 164: 1597, 129: 365, 119: 368, 163: 1
, 72: 217, 86: 240, 130: 355, 55: 164, 98: 293, 172: 677, 168: 1006, 94: 280, 48: 109, 105: 317, 62: 210, 36: 48, 104: 313, 121: 343, 124: 368, 178: 306, 167: 1205,
42: 68, 40: 72, 93: 285, 166: 1241, 116: 373, 103: 307, 66: 240, 46: 76, 83: 237, 80: 227, 174: 517, 128: 366, 126: 395, 113: 317, 57: 172, 58: 189, 75: 216, 170: 10
111: 341, 59: 190, 53: 147, 69: 205, 61: 214, 115: 355, 171: 985, 127: 333, 71: 199, 120: 315, 185: 468, 87: 249, 108: 331, 35: 40, 56: 170, 78: 243, 49: 126, 54: 1
179: 314, 91: 266, 100: 345, 47: 100, 95: 289, 89: 244, 176: 406, 182: 518, 70: 215, 51: 126, 77: 204, 109: 337, 183: 455, 85: 255, 112: 363, 68: 225, 175: 451, 97:
78, 74: 195, 110: 325, 73: 197, 117: 362, 187: 487, 65: 224, 123: 369, 191: 582, 64: 228, 60: 200, 102: 311, 92: 282, 173: 539, 192: 532, 45: 71, 189: 585, 44: 73, 4
561, 197: 436, 181: 457, 193: 514, 63: 213, 96: 290, 208: 379, 207: 373, 201: 476, 196: 518, 213: 415, 199: 443, 206: 423, 188: 467, 107: 352, 106: 329, 210: 393, 1
7: 341, 204: 432, 184: 386, 122: 390, 52: 128, 202: 469, 125: 335, 41: 69, 195: 505, 38: 63, 233: 71, 225: 307, 216: 454, 211: 365, 215: 436, 226: 298, 227: 263, 37
, 209: 382, 214: 376, 234: 61, 235: 46, 224: 357, 218: 375, 231: 94, 244: 10, 32: 46, 217: 484, 221: 381, 219: 376, 222: 345, 212: 364, 229: 175, 230: 116, 223: 367,
```

Paso 5: Deberemos reconstruir la imagen con la transformación de los píxeles.

```
def reconstruir_imagen(gris, valores):
    pendiente, constante = valores
    nueva_imagen = pendiente * gris + constante

    nueva_imagen = np.clip(nueva_imagen, 0, 255)
    return nueva_imagen.astype(np.uint8)
```

Resultado



Explicación del programa

1. Bibliotecas

Figura 8

```
1 import cv2
2 import matplotlib.pyplot as plt
3 import numpy as np
4
```

Nota: Elaboración propia

2. Cantidad_pixeles_color: Cuenta cuántas veces aparece cada nivel de gris en la imagen.

Figura 9

```
6 def cantidad_pixeles_color(lista):
7     elementos = {}
8     for i in lista:
9         if i not in elementos:
10             elementos[i] = 0
11
12         if i in elementos.keys():
13             elementos[i] += 1
14
15     return elementos
16
```

Nota: Elaboración propia.

3. `Dibujar_histograma()`: Utiliza los niveles de gris como eje horizontal y la cantidad de píxeles como eje vertical para representar gráficamente la distribución de intensidades de la imagen en escala de grises.

Figura 10

```

17 def dibujar_histograma(pixeles):
18     niveles = list(pixeles.keys())
19     cantidades = list(pixeles.values())
20
21     plt.bar(niveles, cantidades, width=1)
22
23     plt.title("Histograma de escala de grises")
24     plt.xlabel("Nivel de gris")
25     plt.ylabel("Cantidad de píxeles")
26     plt.show()
27

```

Nota: Elaboración propia

4. `reordenamiento()`: Determina la pendiente y la constante de una función lineal que permite mapear los niveles de gris originales al nuevo intervalo deseado, usando los valores mínimo y máximo de ambas escalas.

Figura 11

```

29 def reordenamiento(inicial, final):
30     punto1 = (inicial[0], final[0]) #interpretalo como (x, y)
31     punto2 = (inicial[1], final[1])
32     pendiente = (punto2[1] - punto1[1]) / (punto2[0] - punto1[0])
33     constante = final[0] - (pendiente * inicial[0])
34     return (pendiente, constante)
35

```

Nota: Elaboración propia

5. `función_trasnsformacion()`: Aplica la transformación lineal a un nivel de gris.

Figura 12

```

37 def funcion_transformacion(valores, gris):
38     #De la forma T(r) = m * r + b
39     new_valor = int((valores[0] * gris) + valores[1])
40     return new_valor
41

```

Nota: Elaboración propia

6. `Obtener_nuevos_valores()`: Aplica la función de transformación a cada nivel de gris del histograma original, generando un nuevo diccionario que representa la distribución de píxeles después de la expansión del histograma.

Figura 13

```

43 def obtener_nuevos_valores(valores, pixeles):
44     new_elementos = {}
45     for i in pixeles:
46         trans = funcion_transformacion(valores, i)
47         new_elementos[trans] = pixeles[i]
48     return new_elementos
49

```

Nota: Elaboración propia

7. `reconstruir_imagen()`: Aplica la función de transformación lineal a cada píxel de la imagen en escala de grises, limitando los valores al rango [0,255], y genera una nueva imagen con mayor contraste.

Figura 14

```

52 def reconstruir_imagen(ggris, valores):
53     pendiente, constante = valores
54     nueva_imagen = pendiente * ggris + constante
55
56     nueva_imagen = np.clip(nueva_imagen, 0, 255)
57     return nueva_imagen.astype(np.uint8)
58

```

Nota: Elaboración propia.

8. `Main()`: Carga la imagen, la convierte a escala de grises, calcula el histograma original, realiza la expansión del histograma mediante una transformación lineal, reconstruye la imagen resultante y muestra tanto los histogramas como las imágenes original y transformada.

Figura 14

```

60 def Main():
61     #Paso 1: cantidad de pixeles de la imagen
62     imagen = cv2.imread("pina.jpg")
63     alto, ancho, canales = imagen.shape
64     print("ancho: ", ancho, "Alto:", alto)
65     #paso 2: Obtenemos la escala de grises en una lista
66     gris = cv2.cvtColor(imagen, cv2.COLOR_BGR2GRAY)
67     niveles_gris = gris.flatten()
68     print(niveles_gris)
69     print(max(niveles_gris), min(niveles_gris))
70

```

Nota: Elaboración propia.

Figura 15

```

71 #Paso 3 diccionario con la cantidad
72 pixeles = cantidad_pixeles_color(niveles_gris)
73 print(pixeles) #observar la escala de gris con la cantidad de píxeles
74 dibujar_histograma(pixeles)
75
76 print("\nValores transformados\n-----\n")
77
78 #Paso 4 Reordenamiento del histograma
79 escala_original = (min(niveles_gris), max(niveles_gris))
80 escala_nueva = (0, 255)
81
82 valores = reordenamiento(escala_original, escala_nueva)
83 print(valores) #Cálculo de la pendiente y constante de la función de transformación
84 new_valores = obtener_nuevos_valores(valores, pixeles)
85 dibujar_histograma(new_valores)
86 print(new_valores) #observar la escala de gris transformada con la cantidad de píxeles
87
88 #Paso 5 Imagen obtenida
89 imagen_expandida = reconstruir_imagen(gris, valores)
90 cv2.imshow("Imagen original", gris)
91 cv2.imshow("Imagen expandida", imagen_expandida)
92 cv2.waitKey(0)
93 cv2.destroyAllWindows()
94
95 #guardar la imagen
96 #cv2.imwrite("pina_expandida.jpg", imagen_expandida)
97
98 #principal
99 Main()

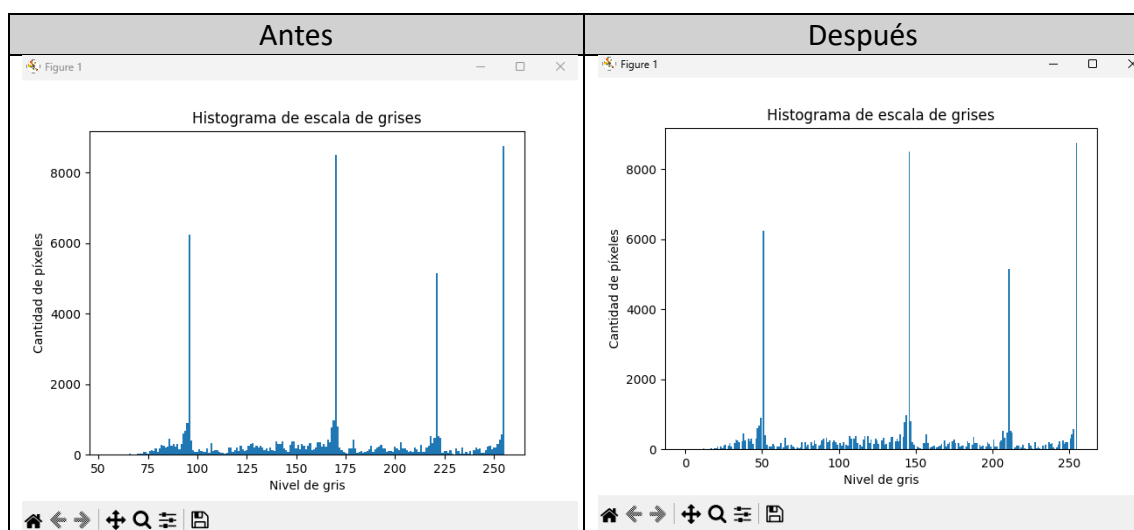
```

Nota: Elaboración propia.

Resultados

Figura 16

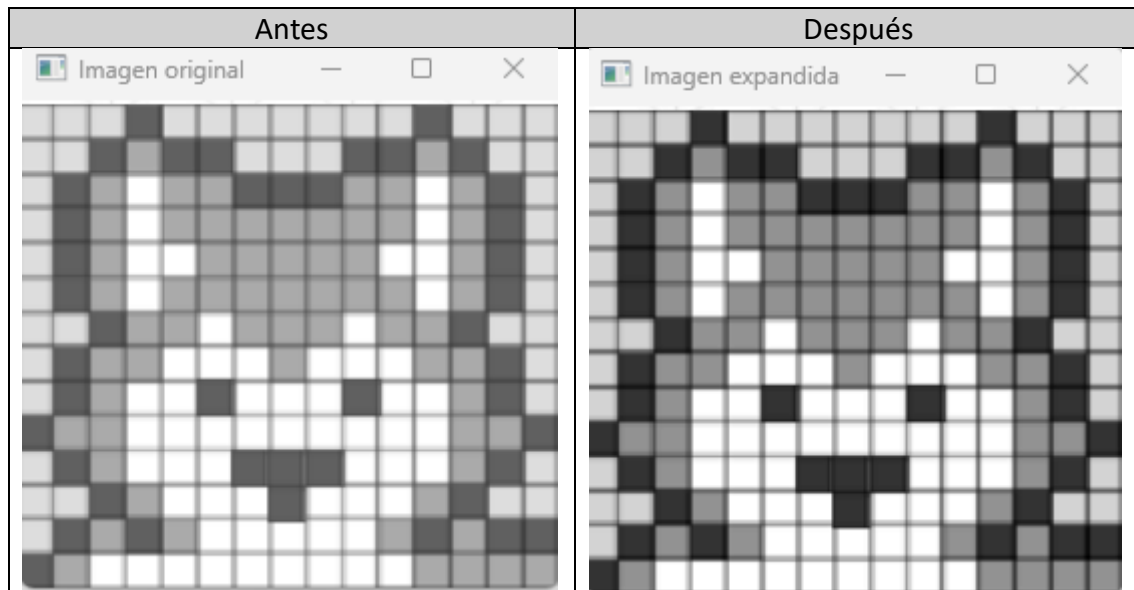
Comparación de histogramas



Nota: Elaborado con Python.

Figura 17

Comparación de imágenes.



Nota: Elaborado con Python.

Conclusiones

La expansión del histograma permitió mejorar el contraste de la imagen al redistribuir los niveles de gris en todo el rango dinámico disponible, facilitando una mejor visualización de los detalles.

La aplicación de una transformación lineal demostró ser un método efectivo y sencillo para el reescalamiento de intensidades en imágenes digitales, manteniendo la relación entre los niveles de gris originales.

El uso de histogramas permitió analizar de manera cuantitativa la distribución de los niveles de gris antes y después del procesamiento, evidenciando el efecto de la mejora del contraste.

El procesamiento digital de imágenes tiene múltiples aplicaciones prácticas, y técnicas como la expansión del histograma constituyen una herramienta fundamental en áreas como la medicina, la industria y la agricultura.

Referencias Bibliográficas

Fernández, L. A., Díaz, D., & Depaoli, R. (2005). Optimización de la ecualización del histograma en el procesamiento de imágenes digitales. In *VII Workshop de Investigadores en Ciencias de la Computación*.

<https://sedici.unlp.edu.ar/handle/10915/21082>

Moreno, W. F., Tangarife, H. I., & Escobar Díaz, A. (2017). *Image analysis applications in precision agriculture*. *Visión Electrónica*, 11(2), 200–210.

<https://revistas.udistrital.edu.co/index.php/visele/article/view/14628>

ANEXOS

Link github: <https://github.com/Ca-mo-nan/C-digos-Matematicos/tree/3335a43ead40898b08d97c3537ad39dd120d946f/math-computing/image-processing-one>